



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Práctica #3: Simular Controlador de Volumen Gráfico”

Computo Integrado

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 135

Hora: N4

Equipo: 5

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
2003996	Attiana del Carmen Barrios Pérez	ITS
2018561	Mariana Cecilia Vaquera Cuellar	ITS
1962995	Carlos Abraham Gallardo Treviño	ITS
2015012	Fernanda Abigail Castillo Morales	ITS
1884405	Sergio Salazar Palacios	ITS
1960381	Andres Javier Alvarado Velez	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 20 de febrero
de 2024

Objetivo de la práctica

Utilizar el Raspberry Pi para controlar tanto los leds como el display al leer los valores del potenciómetro y mapearlos.

Hallazgos o dificultades

Se presentaron problemas al conectar los LEDs, ya que no contábamos con el componente MCP3008 tuvimos que adaptarnos a los materiales que teníamos, sustituyéndolo así por un Arduino nano el cual hace el mismo trabajo del microchip que es convertir la señal analógica a digital, otro problema que presentamos fue que nos faltó conectar el jumper que dirige la tierra del Raspberry asimismo se tuvieron que conectar algunos LEDs con jumper para que funcionara.

Uno de los hallazgos que logramos identificar fue la señal que debía obtener el Raspberry Pi del Arduino, como la señal la convierte a 0 y 1, usamos directamente la compuerta de la conexión e implementamos en el código la conversión.

Código/Sketch

Arduino

```
const int analogPin = A0; // El pin al que está conectado el potenciómetro
const int digitalPin = 2; // El pin digital que enviará el valor al Raspberry Pi

void setup() {
  pinMode(analogPin, INPUT);
  pinMode(digitalPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int analogValue = analogRead(analogPin);

  int digitalValue = map(analogValue, 0, 1023, 0, 254); // Mapear de 0 a 1023 a 0 a 254

  digitalWrite(digitalPin, digitalValue);
  Serial.println(digitalValue);
}
```

```
    delay(100);  
}
```

Raspberry Pi

```
import serial  
import time  
import RPi.GPIO as GPIO
```

```
segmentos = {  
    'A': 2,  
    'B': 3,  
    'C': 4,  
    'D': 17,  
    'E': 27,  
    'F': 22,  
    'G': 10,  
    'DP': 9  
}
```

```
numeros = {  
    0: ['G'],  
    1: ['A', 'D', 'E', 'F', 'G'],  
    2: ['C', 'F'],  
    3: ['F', 'D'],  
    4: ['A', 'D', 'E'],  
    5: ['B', 'D'],  
    6: ['B'],  
    7: ['D', 'E', 'F', 'G'],  
    8: [],  
    9: ['D']  
}
```

```
led_pins = [11, 5, 6, 13, 19, 26, 21, 20, 16]
```

```
# Configuración de la comunicación serial  
ser = serial.Serial('/dev/ttyUSB0', 9600)  
ser.flush()
```

```
GPIO.setmode(GPIO.BCM)  
for seg_pin in segmentos.values():
```

```

GPIO.setup(seg_pin, GPIO.OUT)
GPIO.output(seg_pin, GPIO.LOW)
for led_pin in led_pins:
    GPIO.setup(led_pin, GPIO.OUT)
    GPIO.output(led_pin, GPIO.LOW)

try:
    while True:
        line = ser.readline().decode('utf-8').rstrip()

        if line:
            valor_analogico = int(line)

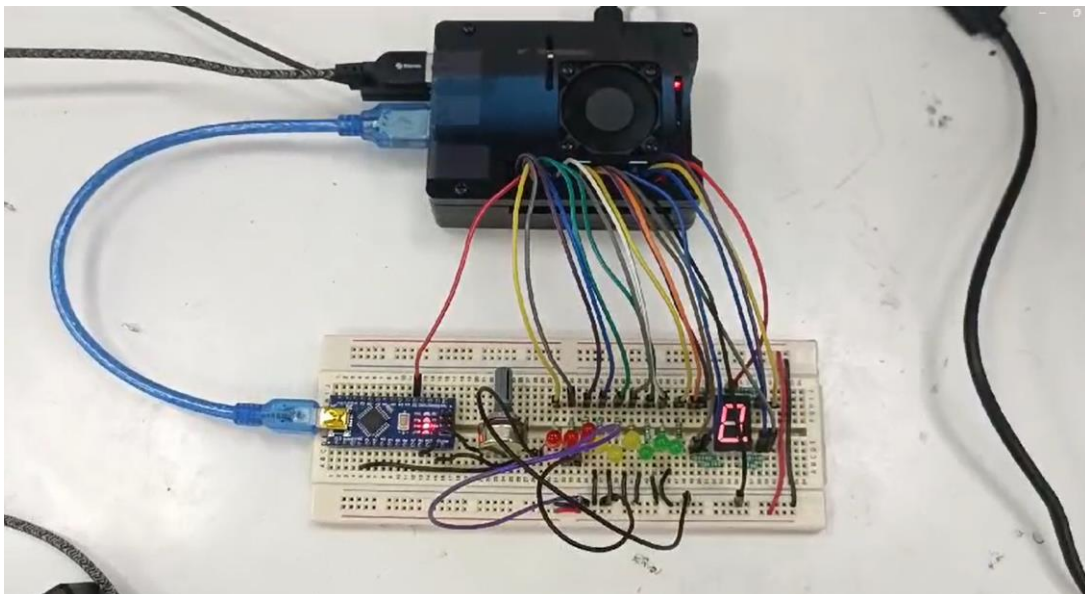
            valor_display = max(0, min(9, valor_analogico // 10))

            for segmento, seg_pin in segmentos.items():
                if segmento in numeros[valor_display]:
                    GPIO.output(seg_pin, GPIO.HIGH)
                else:
                    GPIO.output(seg_pin, GPIO.LOW)

            for i in range(valor_display):
                GPIO.output(led_pins[i], GPIO.HIGH)
            for i in range(valor_display, len(led_pins)):
                GPIO.output(led_pins[i], GPIO.LOW)

            time.sleep(0.01)
except KeyboardInterrupt:
    ser.close()
    GPIO.cleanup()

```



Conclusión general

En esta práctica, el equipo contó con diferentes problemas, la mayoría no teníamos suficiente conocimiento para el armado del prototipo, por lo cual recurrimos a investigar y guiarnos mediante videos e imágenes, tratamos de realizar pruebas en cada paso que teníamos, de esta manera podríamos darnos cuenta de cuáles eran estos errores que se nos presentaban.

Respecto al material que se necesitaba solo nos faltaba un convertidor de analógico a digital, pero este se pudo sustituir por un Arduino nano, todo el equipo terminó aprendiendo cosas nuevas y la práctica se concluyó con éxito.

Conclusiones individuales

<https://youtu.be/73q-ivn53fg>

Video del funcionamiento

<https://youtu.be/iOg7O4k3Et0>

Bibliografía

330ohms. (2020). Cómo conectar un Potenciómetro a Raspberry Pi. 330ohms. Recuperado el 20 de febrero de 2024 del sitio web: <https://blog.330ohms.com/2020/05/06/como-conectar-un-potenciometro-a-raspberry-pi/>

Cytron Technologies. (2018). I2C Raspberry Pi Arduino - Reading Analog Signal [BM] [YouTube Video]. En YouTube. Recuperado el 20 de febrero de 2024 del sitio web: https://www.youtube.com/watch?v=ID_SMeUJHw8&ab_channel=CytronTechnologies

Cytron Technologies. (2018). I2C Communication Between Raspberry Pi and Arduino [BM] [YouTube Video]. En YouTube. Recuperado el 20 de febrero de 2024 del sitio web: https://www.youtube.com/watch?v=zoxcNJr4-t4&ab_channel=CytronTechnologies

BBOYHO & SANTAIMPERSONATOR. (2018). How to Install CH340 Drivers - SparkFun Learn. Sparkfun.com. Recuperado el 20 de febrero de 2024 del sitio web: <https://learn.sparkfun.com/tutorials/how-to-install-ch340-drivers/all#troubleshooting>

Mauricio, O. (2020). Pic c compiler display 7 segmentos cátodo común | CódigoElectronica. Codigoelectronica.com. Recuperado el 20 de febrero de 2024 del sitio web: <http://codigoelectronica.com/blog/pic-c-compiler-display-7-segmentos-catodo-comun>

Abellan, M. (2019). Primeros pasos con Pines GPIO en Raspberry Pi. Asociación Programo Ergo Sum. Recuperado el 20 de febrero de 2024 del sitio web: <https://www.programoergosum.es/tutoriales/introduccion-a-pines-gpio-en-raspbian/>